

```
Mystruct s, *ps, arrs[10];
```

Usually, you don't need both tag and typedef: either can be used in structure type declarations.

Untagged structure and union members are ignored during initialisation.

4.2.6 Types conversions

C is strictly typed language, with each operator, statement and function demanding appropriately typed operands/arguments. However, we often have to use objects of “mismatching” types in expressions. In that case, type conversion is needed.

Conversion of object of one type is changing it to the same object of another type (i.e. applying another type to a given object). C defines a set of standard conversions for built-in types, provided by compiler when necessary

Conversion is required in following situations:

- if statement requires an expression of particular type (according to language definition), and we use an expression of different type,

- if operator requires an operand of particular type, and we use an operand of different type,

- if a function requires a formal parameter of particular type, and we pass it an object of different type,

- if an expression following the keyword return does not match the declared function return type,

In these situations, compiler will provide an automatic implicit conversion of types, without any user interference. Also, user can demand conversion explicitly by means of typecast operator.

4.2.7 Declarations

Declaration introduces one or several names to a program – it informs the compiler what the name represents, what is its type, what are allowed operations with it, etc. This section reviews concepts related to declarations: declarations, definitions, declaration specifiers, and initialization.

The range of objects that can be declared includes:

- Variables

- Constants

- Functions

- Types

- Structure, union, and enumeration tags

- Structure members